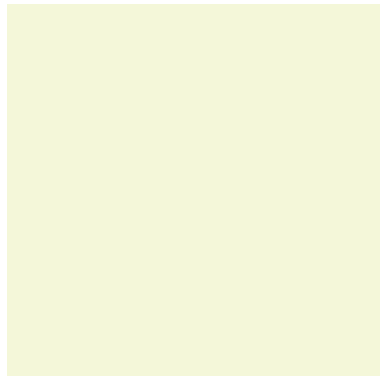


Comprehensive Python Programming



Agenda

- Intro to Python
- Installing & Setup
- Python Basics
- Data Types & Functions
- String & Sequence Ops
- Built-in Functions & math
- Python Operators
- Conditional Statements
- Loops & Controls
- List Data Structure
- Advanced Data Structures
- Comprehensions
- Functions & Arguments
- Advanced Functional Programming

- Modules & Packages
- Regular Expressions
- Logging Basics
- File Handling
- CSV Files
- SQLite Intro
- OOP Basics
- Advanced OOP
- Inheritance & Polymorphism
- Method Overloading & Access
- Exception Handling
- Debugging Tips
- Mini Projects
- Recap & Challenges

Introduction to Python

What is Python?

Python is a high-level, interpreted programming language known for its clear syntax and readability, making it ideal for beginners and experts alike.

Why Python?

Python's simplicity, extensive libraries, and strong community support make it a preferred choice for rapid development and diverse programming tasks.

Applications

Python is used in web development, data science, artificial intelligence, automation, scientific computing, and more, showcasing its versatility.

Features

Key features include dynamic typing, automatic memory management, a large standard library, and support for multiple programming paradigms including procedural, object-oriented, and functional programming.

Installing Python & Environment Setup

Installing Python on Windows, Linux, Mac

Download the latest Python installer from the official website. On Windows, run the executable and ensure 'Add Python to PATH' is checked. Linux users can install via package managers like apt or yum. Mac users can use the official installer or Homebrew.

Setting Environment Variables

Configure environment variables like PATH to allow running Python commands from the terminal. On Windows, update system variables via Control Panel. On Linux/Mac, modify shell configuration files like .bashrc or .zshrc to export paths.

Choosing and Setting Up IDEs

Popular Python IDEs include PyCharm, VS Code, and Jupyter Notebook. Install and configure your preferred IDE with necessary Python extensions and interpreters for efficient coding and debugging.

Python Basics: Syntax and Input/Output

Syntax Rules

Python uses clear, readable syntax with strict indentation to define code blocks instead of braces, emphasizing code readability and simplicity.

Commenting

Single-line comments start with #, while multi-line comments use triple quotes (""" or """) for documenting code and improving clarity.

Indentation

Indentation is mandatory in Python to define blocks of code like loops and functions; consistent use of spaces or tabs is required.

Printing Output

The print() function outputs text or variables to the console, supporting string formatting and multiple arguments for flexible display.

Input Function

The input() function allows user input from the console as a string, which can be converted to other types for interactive programs.

Data Types and Built-in Functions

Core Data Types

Python's fundamental data types include `int` (integer numbers), `float` (decimal numbers), `str` (text strings), and `bool` (True/False values). These types form the basis of data representation in Python.

Type Inspection Functions

Functions like `type()` help identify the data type of a variable. Using `help()` provides documentation for functions and objects, while `dir()` lists the attributes and methods available for an object.

Practical Usage Tips

Use `int()`, `float()`, `str()`, and `bool()` to convert between types explicitly. Leverage `help()` to understand unfamiliar functions and `dir()` to explore object capabilities during development.

String & Sequence Operations

Built-in String Methods

Python provides numerous built-in string methods such as `.lower()`, `.upper()`, `.strip()`, `.replace()`, and `.split()` to manipulate and transform string data easily.

Slicing Sequences

Slicing extracts a subsequence using the syntax `[start:stop:step]`, enabling selection of parts of strings, lists, or tuples efficiently.

Indexing & Negative Indexing

Indexing allows access to individual elements in a sequence starting at 0, while negative indexing accesses elements from the end, starting at -1 for the last element.

Applications & Examples

Use slicing to reverse strings, extract substrings, or iterate over sequences selectively; combined with methods for powerful data handling.

Common Built-in Functions & math Module

± min() and max()

min() returns the smallest item in an iterable or among multiple arguments, while max() returns the largest, useful for quick comparisons.

×² abs() and pow()

abs() gives the absolute value of a number, removing any negative sign. pow() raises a number to a specified power, supporting modular arithmetic.

🔗 round()

round() rounds a floating-point number to the nearest integer or specified decimal places, helping in formatting and precision control.

📊 math module basics

The math module provides advanced mathematical functions like sqrt(), factorial(), sin(), cos(), and constants such as pi and e for scientific calculations.

√ Using math.sqrt() and math.factorial()

math.sqrt() computes the square root efficiently, while math.factorial() calculates the factorial of a non-negative integer, both essential in algorithms.

π Constants and trigonometry

math.pi and math.e are fundamental constants. Trigonometric functions such as sin(), cos(), and tan() enable handling of angles and periodic phenomena.

Operators in Python

Arithmetic Operators

Include addition (+), subtraction (-), multiplication (*), division (/), modulus (%), exponentiation (**), and floor division (//) used for numerical calculations.

Conditional Operators

Comparison operators such as ==, !=, >, <, >=, <= evaluate relationships between values and return Boolean outcomes.

Logical Operators

Operators like and, or, not are used to combine conditional statements and return Boolean results based on logical conditions.

Bitwise Operators

Operate on bits of integers using &, |, ^, ~, <<, >> to perform AND, OR, XOR, NOT, left shift, and right shift operations.

Assignment Operators

Used to assign values to variables, including =, +=, -=, *=, /=, %=, **=, and //= which combine assignment with arithmetic operations.

Membership & Identity

'in' and 'not in' check for membership in sequences, while 'is' and 'is not' test object identity in memory.

Conditional Statements

Simple if Statement

Executes a block of code only if a specified condition evaluates to True. It allows basic decision-making by checking a single condition.

Syntax example: if condition:
code block

Use cases: Validate user input, conditional execution

if-else Statement

Provides two branches: one executes if the condition is True, and the other executes if it is False, enabling binary decisions.

Syntax example: if condition:
code block

else:
alternative code
Use cases: Toggle features, handle success/failure scenarios

if-elif-else Ladder

Handles multiple conditions by chaining if and elif statements, with else as a fallback, supporting complex decision trees.

Syntax example: if condition1:
code block

elif condition2:
code block
else:
fallback code
Use cases: Grade classification, multi-choice logic

Ternary Operator

A concise, inline conditional expression that returns one of two values based on a condition, improving code readability for simple decisions.

Syntax example: value_if_true if condition else value_if_false
Use cases: Assign default values, conditional expressions in one line

Loops and Control Statements

For Loop

Executes a block of code repeatedly for each item in a sequence or iterable, such as a list or range.

For loop syntax
Iterating over lists
Using range() with for loops

Range() Function

Generates a sequence of numbers used commonly with for loops to specify the iteration count or a sequence.

range(start, stop, step)
examples
Using range for numeric loops
Creating custom sequences

While Loop

Repeats a block of code as long as a given condition remains True, suitable for indefinite iteration.

While loop syntax
Implementing loops with conditions
Avoiding infinite loops

Break Statement

Terminates the nearest enclosing loop immediately, allowing early exit based on a condition.

Using break in loops
Conditional break examples
Exiting loops prematurely

Continue Statement

Skips the current iteration and moves to the next loop cycle, useful for bypassing certain conditions.

Using continue to skip iterations
Examples with if conditions
Controlling loop flow dynamically

Pass Statement

A placeholder that does nothing and allows code structure to be maintained where a statement is syntactically required.

Using pass in loops and functions
Placeholder for future code
Maintaining loop structure

List Data Structure

	List Creation	Lists can be created using square brackets with comma-separated values, e.g., <code>[1, 2, 3]</code> , supporting mixed data types like integers, strings, and more.
	Common List Methods	Key methods include <code>append()</code> to add items, <code>remove()</code> to delete, <code>pop()</code> to extract by index, <code>insert()</code> to place at a position, and <code>sort()</code> to order items.
	Slicing Lists	Slicing extracts portions using <code>start:stop:step</code> syntax, e.g., <code>list[1:4]</code> , allowing negative indices for reverse selection and efficient sublist creation.

Advanced Data Structures: Tuple, Dictionary, Set

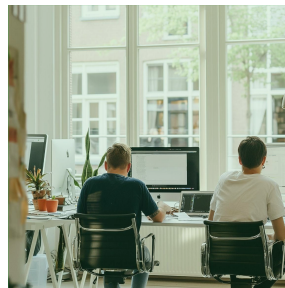
**Tuples: Packing,
Unpacking,
Immutability**



**Dictionaries:
Key-Value Pairs
& Methods**



**Sets: Creation,
Operations,
Methods**



Tuples are immutable sequences; you can pack multiple values into one tuple and unpack them back into variables with ease.

Dictionaries store data in key-value pairs and provide powerful methods like `.get()`, `.keys()`, `.values()`, and `.items()` for flexible data access and manipulation.

Sets hold unique elements without order, supporting operations like union, intersection, difference, and methods such as `.add()`, `.remove()`, and `.clear()`.

Comprehensions in Python

Comprehensions in Python are elegant, compact syntaxes used to create new lists, dictionaries, or sets from existing iterables.

List comprehensions use square brackets and enable filtering and transforming elements with a concise expression, e.g., `[x*2 for x in range(5)]`.

Dictionary comprehensions create key-value pairs dynamically, e.g., `{x: x**2 for x in range(5)}`.

Set comprehensions build unique collections with curly braces, e.g., `{x for x in range(5) if x % 2 == 0}`.

These comprehensions improve code readability and performance by replacing verbose loops and conditionals with a single expressive line.

Functions and Arguments



Defining Functions

Use the 'def' keyword followed by the function name and parentheses. Functions encapsulate reusable code blocks that improve organization and readability.



() Types of Arguments

Python supports positional, keyword, default, and variable-length arguments allowing flexible function calls and parameter handling.



Calling Functions

Invoke functions by using their name followed by parentheses, passing required arguments inside. This executes the code within the function.



Argument Examples

Positional: arguments passed in order; Keyword: specify by name; Default: assigned if no argument passed; Variable-length: *args for tuples, **kwargs for dictionaries.

Advanced Functional Programming

Lambda Functions

Anonymous, small, and inline functions defined with the lambda keyword; useful for short, throwaway functions without formal definition.

Map, Filter, Reduce

Map applies a function to all items in a list; filter selects items based on a condition; reduce aggregates items into a single value using a function.

Iterators & Generators

Iterators allow sequential access to elements; generators produce items lazily using yield, enabling efficient memory use for large datasets.

Decorators

Functions that modify or extend behavior of other functions or methods without changing their code, typically used for logging, access control, or memoization.

Benefits & Use Cases

Advanced functional tools improve code modularity, reusability, and functional purity; commonly used in data processing, event handling, and API design.

Modules and Packages



Importing Standard Modules

Use import statements to include built-in modules like datetime for date/time, os for system operations, sys for interpreter controls, random for randomization, math for mathematical functions, and string for string operations.



Packages for Organization

Packages are directories with `__init__.py` files that group related modules. They help structure large projects by logically organizing code into namespaces.



Creating Custom Modules

Write reusable Python code in separate .py files. Import these custom modules into other scripts to modularize and maintain your projects effectively.



Special Variable `__name__`

Use `if __name__ == '__main__':` to ensure code runs only when the module is executed directly, not when imported, enabling modular testing and script reuse.

Regular Expressions in Python

The `re` module in Python is used for working with regular expressions (regex), which allow complex pattern matching within strings. Wildcards like `'.'` match any character, while meta characters such as `'^'` (start of string), `'$'` (end of string), `'*'` (zero or more repetitions), `'+'` (one or more repetitions), and `'?'` (optional) help define flexible search patterns.

Python's `re` module offers key methods including `re.match()` to check for a match at the beginning of a string, `re.search()` to find a match anywhere in the string, `re.findall()` to retrieve all matches, and `re.sub()` to substitute matches with a replacement string.

These tools simplify tasks like validation, parsing, and text manipulation by providing concise and efficient ways to work with string patterns in Python.

Logging in Python

Logging Basics

Python's logging module provides a flexible framework to record events, errors, and informational messages during application execution, aiding in debugging and monitoring.

Log Levels

Standard log levels include DEBUG, INFO, WARNING, ERROR, and CRITICAL, allowing developers to filter messages based on their severity and importance.

Log File Management

Logs can be written to files for persistent storage and later analysis, helping track application behavior over time and diagnose issues.

Formatters

Formatters define the structure and content of log messages, including timestamps, log levels, module names, and the actual message for clear and consistent output.

Handlers

Handlers determine where log messages are sent, such as the console, files, or external systems, enabling flexible routing and multi-destination logging.

File Handling



Opening and Closing Files

Use the `open()` function with modes like 'r' for read, 'w' for write, 'a' for append. Always close files with `close()` or use 'with' to auto-manage.



Reading and Writing Files

Read file contents using `read()`, `readline()`, or `readlines()`. Write data with `write()` or `writelines()` methods to store or update file content.



Using 'with' Keyword and File Functions

'with' ensures files close automatically after operations. File functions include `seek()`, `tell()` for cursor control, enhancing file handling safety and efficiency.

Working with CSV Files

Reading CSV Files

- Use `csv.reader()` to read CSV files row by row.
- Open files with 'r' mode and `newline=""` for compatibility.
- Iterate over reader object to access each row as a list.
- Handle headers separately if present to map data correctly.
- Example: `for row in reader: process(row)`

Writing CSV Files

- Use `csv.writer()` to write rows to CSV files.
- Open files with 'w' mode and `newline=""` to avoid blank lines.
- `writerow()` writes a single row; `writerows()` writes multiple rows.
- Common to write headers first for structured data files.
- Example: `writer.writerow(['Name', 'Age', 'City'])`

Introduction to SQLite with Python

SQLite is a self-contained, serverless database engine widely used for embedded applications. Python's built-in 'sqlite3' module allows seamless integration without extra setup.

Connect to a database using `sqlite3.connect()`, which creates the file if missing. Use `connection.cursor()` to create a cursor for executing SQL queries.

Execute SQL commands with `cursor.execute()`, save changes with `connection.commit()`, and get the last inserted row ID using `cursor.lastrowid`. This makes SQLite ideal for lightweight data storage in Python apps.

Object-Oriented Programming Basics



Class and Object

A class is a blueprint for creating objects. An object is an instance of a class that encapsulates data and behavior.



The self Parameter

The self parameter represents the instance of the class and allows access to its attributes and methods within class definitions.



The __init__ Method

The __init__ method initializes newly created objects, setting initial attribute values when an instance is created.

Advanced OOP Concepts

setattr Function

Used to dynamically set or modify an attribute of an object. Allows flexible attribute management during runtime, e.g., `setattr(obj, 'attr', value)`.

hasattr Function

Checks if an object has a specific attribute, returning True or False. Useful for safe attribute access and conditional logic.

delattr Function

Deletes an attribute from an object dynamically. Helps in managing object state by removing unnecessary or sensitive attributes.

classmethod Decorator

Defines a method bound to the class rather than its instance. It receives the class as the first argument and is used for factory methods or alternative constructors.

staticmethod Decorator

Defines a method that neither takes the instance nor the class as the first argument. Used for utility functions related to the class but not dependent on instance or class state.

Inheritance and Polymorphism

Types of Inheritance in Python

- Single inheritance: One child class inherits from a single parent class.
- Multiple inheritance: A child class inherits from more than one parent class.
- Multilevel inheritance: A class inherits from a derived class, creating a chain of inheritance.
- Hierarchical inheritance: Multiple child classes inherit from a single parent class.
- Hybrid inheritance: Combination of two or more types of inheritance in a single program.

Polymorphism Examples

- Method overriding allows a subclass to provide a specific implementation of a method already defined in its parent class.
- Operator overloading lets operators behave differently based on operand types, e.g., using '+' for both numbers and strings.
- Duck typing in Python enables polymorphism by focusing on methods and properties rather than object types.
- Example: Different classes implementing a common method name with different behaviors called through the same interface.
- Polymorphism improves code flexibility, making it easier to extend and maintain.

Method Overloading & Access Modifiers



Method Overloading in Python

- Python supports method overloading by default using default arguments.
- True method overloading (same method name, different parameters) is not natively supported.
- Use `*args` and `**kwargs` to handle variable argument lists in a single method.
- Example: Define multiple behaviors in a method based on argument count or type.



Access Modifiers: Public, Private, Protected

- Python uses naming conventions to indicate access levels, not enforced by compiler.
- Public: Accessible from anywhere, no underscore prefix (e.g., `my_var`).
- Protected: Indicated by a single underscore prefix (e.g., `_my_var`), meant for internal use.
- Private: Double underscore prefix (e.g., `__my_var`) triggers name mangling to restrict access.

Exception Handling

try-except-finally Structure

Use try block to wrap code that may raise errors; except handles those errors gracefully; finally executes cleanup code regardless of exceptions.

Common Exceptions

Frequent exceptions include ValueError, TypeError, IndexError, KeyError, and ZeroDivisionError, each indicating specific runtime issues.

User-Defined Exceptions

Create custom exception classes by inheriting from Exception to represent application-specific error conditions for clearer error handling.

Raising Exceptions

Use the raise statement to trigger exceptions manually, allowing controlled error signaling and enhanced debugging.

Debugging and Messaging

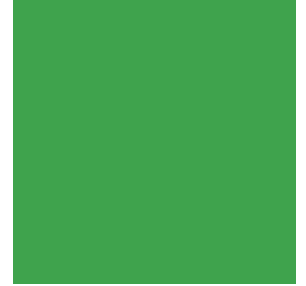
Importance of Messaging

- Clear messages help identify issues quickly in code.
- Logging messages provide context for errors and program flow.
- Consistent messaging aids collaboration and future debugging.
- Use informative messages for both normal and error states.

Debugging Tips

- Use print statements or logging to trace code execution.
- Leverage debugging tools like breakpoints and step execution.
- Isolate problems by testing smaller code sections.
- Check variable values and program state during runtime.
- Understand common error types and their causes.

Mini Projects Overview



File Parser

Develop a program to read, interpret, and extract meaningful data from various file formats, enhancing file handling and text processing skills.

Log Analyzer

Create a tool to process and analyze log files for patterns, errors, and summaries to improve debugging and monitoring capabilities.

Database CRUD Application

Build a simple app to Create, Read, Update, and Delete records in a SQLite database, demonstrating database connectivity and query execution.

Recap and Final Coding Challenges

- Review key Python concepts: data types, control structures, functions, and OOP fundamentals.
- Apply problem-solving strategies: break down problems, write pseudocode, and test incrementally.
- Focus on debugging techniques: use print statements, logging, and Python debugging tools effectively.
- Engage with final coding challenges that combine multiple topics for comprehensive practice.
- Use challenges to build confidence and prepare for real-world Python programming tasks.